

Spark Homes — Repair Estimator

Developer Contest Submission Writeup
June 2026

1. MOST INTERESTING DESIGN DECISION

The Deal Analyzer as the true core feature. The brief asked for a repair estimator, but the real job agents need done at the end of a walkthrough is answering one question: *does this deal work at this price?* Repair cost is an input, not the output.

The Deal Analyzer takes ARV, purchase price, and holding costs alongside the live repair total and instantly surfaces: projected profit, ROI, deal health (Strong / Marginal / Weak), and the **Max Allowable Offer** using the 70% rule ($ARV \times 70\% - \text{Repairs} - \text{Holding}$). If the purchase price already exceeds the MAO, a red warning appears immediately.

The UX decision was to make this one tap from every project — not buried in a settings menu — because agents need it at the moment they're deciding whether to make an offer. The share button copies a plain-text summary to the clipboard for quick texting to the acquisition manager.

2. WHAT IS BROKEN OR FRAGILE

Issue	Impact	Fix with more time
OCR requires network	Fails silently offline; user must add serial # manually	Cache the model or use on-device ML (Tesseract WASM)
Photos in localStorage	5–10MB browser quota hit on 5+ high-res photos	Move to IndexedDB or upload to S3
SW caches app shell only	CDN scripts (Tailwind, XLSX, JSZip) need first-load network	Self-host or inline all CDN assets
No multi-device sync	Two agents on same project = last-write-wins collision	Add a lightweight backend (Supabase / Cloudflare KV)

3. CREATIVE ADDITION — DEAL ANALYZER

Every Spark Homes deal lives or dies on the math. The repair estimate is only half the equation — agents also need to know if the purchase price leaves room for profit before they submit an offer.

Why I chose it:

The brief described agents standing in an empty house trying to answer "how much will it cost to fix this?" But experienced flippers know the harder question is "what's the most I can pay and still make money?" The 70% rule is the industry-standard heuristic for exactly this. Building it into the estimator turns a cost-tracking tool into a deal decision tool.

How it works:

Agent enters ARV and purchase price (holding costs optional). The app calculates: $\text{Total Cost Basis} = \text{Purchase} + \text{Repairs} + \text{Holding}$. $\text{Profit} = \text{ARV} - \text{Total Cost Basis}$. $\text{MAO} = \text{ARV} \times 0.70 - \text{Repairs} - \text{Holding}$. If $\text{Purchase} > \text{MAO}$, a warning fires. Deal health is color-coded green/yellow/red by margin %.

4. WHAT I'D SHIP NEXT WITH 2 MORE DAYS

IndexedDB photo storage: Replace localStorage base64 with IndexedDB blobs. Removes the quota ceiling that limits photos per project.

Shareable project URL: Encode project state into a compressed URL hash. Agents can share an estimate link with the office without emailing files.

Contractor quick-dial: Per-trade contact list: tap "HVAC" to see saved contractor names and numbers. Fills the gap between "what needs fixing" and "who do I call."

Offline CDN bundle: Inline Tailwind (CSS only, ~15KB), XLSX.js, and JSZip into the HTML. Makes the app fully air-gapped for basements and rural properties.

AI TOOLS DISCLOSURE

This app was built using **Claude (Anthropic)**. Claude wrote the full implementation — data model, all render functions, the export pipeline, and the Deal Analyzer — based on the brief and price list. Architecture decisions, UX direction, and feature prioritization were directed by the submitter with iterative feedback. Claude also powers the in-app serial number OCR feature at runtime via the Vision API.